

labassignment9_new

April 6, 2025

1 Lab Assignment 9: Data Management Using pandas, Part 2

1.1 DS 6001: Practice and Application of Data Science

1.1.1 Robert Clay Harris jbm2rt

1.1.2 Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

In this lab, we are going to build the Country Analysis Relational DataBase (which we will call the C.A.R.D.B. or the “Cardi B”):

We will be collecting data from two sources. First, we will use open data from the World Bank’s [Sovereign Environmental, Social, and Governance \(ESG\) Data](#) project. The ESG data reports data from every country in the world over the time frame from 1960-2022 on a wide variety of topics including education, health, and economic factors within the countries. Second, we will use data on the quality and democratic character of countries’ governments as reported by the [Varieties of Democracy \(V-Dem\)](#) project at the University of Notre Dame. By using both data sources, we can conduct analyses to see whether democratic openness leads to better societal outcomes for countries. We can also write queries to capture a wide range of information on countries’ political parties, tax systems, and banking industries, for example. Or as Cardi B would say, “You in the club just to party, I’m there, I get paid a fee. I be in and out them banks so much, I know they’re tired of me.”

1.2 Problem 0

Import the following packages (use `pip install` to download any packages you don’t already have installed):

```
[2]: import numpy as np
import pandas as pd
import requests
import os
import io
import zipfile
```

Both the World Bank and V-Dem store their data in zipped directories containing CSV files. Download the World Bank data into your current working directory by typing the following code:

```
url = 'https://databank.worldbank.org/data/download/ESG_CSV.zip' r = requests.get(url) z =
zipfile.ZipFile(io.BytesIO(r.content)) z.extractall()
```

And download the V-Dem data by typing:

```
url = 'https://v-dem.net/media/datasets/V-Dem-CY-Core_csv_v13.zip' r = requests.get(url) z =
zipfile.ZipFile(io.BytesIO(r.content)) z.extractall()
```

After you've run this code successfully once, the files you need will be in your working directory and you should save time by switching these cells from "code" to "raw" so that they don't run again if you restart the kernel.

You will only need two of the files you've downloaded. Load the 'V-Dem-CY-Core-v13.csv' file as `vdem` and the 'ESGData.csv' file as `wb`.

```
[3]: vdem = pd.read_csv('V-Dem-CY-Core-v13.csv')
wb = pd.read_csv('ESGCSV.csv')
```

```
[4]: wb
```

```
[4]: Country Name Country Code \
0 Arab World ARB
1 Arab World ARB
2 Arab World ARB
3 Arab World ARB
4 Arab World ARB
...
16964 Zimbabwe ZWE
16965 Zimbabwe ZWE
16966 Zimbabwe ZWE
16967 Zimbabwe ZWE
16968 Zimbabwe ZWE

Indicator Name Indicator Code \
0 Access to clean fuels and technologies for coo... EG.CFT.ACCS.ZS
1 Access to electricity (% of population) EG.ELC.ACCS.ZS
2 Adjusted savings: natural resources depletion ... NY.ADJ.DRES.GN.ZS
3 Adjusted savings: net forest depletion (% of GNI) NY.ADJ.DFOR.GN.ZS
4 Agricultural land (% of land area) AG.LND.AGRI.ZS
...
16964 Terrestrial and marine protected areas (% of t... ER.PTD.TOTL.ZS
16965 Tree Cover Loss (hectares) AG.LND.FRLS.HA
16966 Unemployment, total (% of total labor force) (... SL.UEM.TOTL.ZS
16967 Unmet need for contraception (% of married wom... SP.UWT.TFRT
16968 Voice and Accountability: Estimate VA.EST

1960 1961 1962 1963 1964 1965 ... \
```

0	NaN	NaN	NaN	NaN	NaN	NaN	...
1	NaN	NaN	NaN	NaN	NaN	NaN	...
2	NaN	NaN	NaN	NaN	NaN	NaN	...
3	NaN	NaN	NaN	NaN	NaN	NaN	...
4	NaN	30.981414	30.982663	31.007054	31.018001	31.042466	...
...	...	...	...	...	...	...	...
16964	NaN	NaN	NaN	NaN	NaN	NaN	...
16965	NaN	NaN	NaN	NaN	NaN	NaN	...
16966	NaN	NaN	NaN	NaN	NaN	NaN	...
16967	NaN	NaN	NaN	NaN	NaN	NaN	...
16968	NaN	NaN	NaN	NaN	NaN	NaN	...

	2014	2015	2016	2017	2018	2019	\
0	86.705717	86.942778	87.228705	87.390856	87.617862	87.798740	
1	88.832276	89.053852	89.539016	90.662754	89.176939	90.352802	
2	10.050554	6.130655	5.265859	6.245422	8.187714	7.234436	
3	0.084361	0.096672	0.092911	0.102684	0.057123	0.064516	
4	39.834421	39.872575	39.937814	39.984452	39.969738	39.907031	
...	...	...	...	...	...	...	
16964	NaN	NaN	27.214542	27.214585	27.214585	27.214747	
16965	NaN	NaN	NaN	NaN	NaN	NaN	
16966	4.770000	5.412000	5.918000	6.349000	6.767000	7.370000	
16967	10.382129	10.400000	NaN	NaN	NaN	NaN	
16968	-1.256889	-1.166087	-1.178460	-1.195905	-1.136798	-1.163669	

	2020	2021	2022	2023
0	87.948264	88.092536	NaN	NaN
1	90.635050	90.845661	NaN	NaN
2	4.598506	NaN	NaN	NaN
3	0.075686	NaN	NaN	NaN
4	39.973290	39.970742	NaN	NaN
...	...	...	...	...
16964	27.214747	27.214747	27.214747	NaN
16965	NaN	NaN	NaN	NaN
16966	8.651000	9.540000	9.256000	9.116
16967	NaN	NaN	NaN	NaN
16968	-1.113408	-1.135830	-1.102206	NaN

[16969 rows x 68 columns]

1.3 Problem 1

First, let's focus on the `vdem` data ('V-Dem-CY-Core-v13.csv'). Use `pandas` methods to perform the following tasks:

1.3.1 Part a

Keep only the 'country_text_id', 'country_name', 'year', 'v2x_polyarchy', and 'v2peedueq' columns. [1 point]

```
[5]: vdem_subset = vdem[['country_text_id', 'country_name', 'year', 'v2x_polyarchy', 'v2peedueq']]

vdem_subset.head()
```

```
[5]:   country_text_id country_name  year  v2x_polyarchy  v2peedueq
0                MEX      Mexico  1789           0.028         NaN
1                MEX      Mexico  1790           0.028         NaN
2                MEX      Mexico  1791           0.028         NaN
3                MEX      Mexico  1792           0.028         NaN
4                MEX      Mexico  1793           0.028         NaN
```

1.3.2 Part b

Use the .query() method to keep only the rows in which year is greater than or equal to 1960 and less than or equal to 2021. [1 point]

```
[6]: vdem_subset = vdem_subset.query("year >= 1960 and year <= 2021")

vdem_subset.head()
```

```
[6]:   country_text_id country_name  year  v2x_polyarchy  v2peedueq
171                MEX      Mexico  1960           0.232        -1.438
172                MEX      Mexico  1961           0.234        -1.438
173                MEX      Mexico  1962           0.233        -1.438
174                MEX      Mexico  1963           0.233        -1.438
175                MEX      Mexico  1964           0.231        -1.438
```

1.3.3 Part c

Rename 'country_text_id' to 'country_code', 'country_name' to 'country_name_vdem', 'v2x_polyarchy' to 'democracy', and 'v2peedueq' to 'educational_equality'. [1 point]

```
[7]: vdem_subset.rename(columns={
      'country_text_id': 'country_code',
      'country_name': 'country_name_vdem',
      'v2x_polyarchy': 'democracy',
      'v2peedueq': 'educational_equality'
    }, inplace=True)

vdem_subset.head()
```

```
[7]:   country_code country_name_vdem  year  democracy  educational_equality
171          MEX          Mexico  1960       0.232             -1.438
```

172	MEX	Mexico	1961	0.234	-1.438
173	MEX	Mexico	1962	0.233	-1.438
174	MEX	Mexico	1963	0.233	-1.438
175	MEX	Mexico	1964	0.231	-1.438

1.3.4 Part d

Sort the rows by 'country_code' and 'year' in ascending order. [1 point]

```
[8]: vdem_subset.sort_values(by=['country_code', 'year'], inplace=True)

vdem_subset.head()
```

```
[8]:      country_code country_name_vdem year  democracy  educational_equality
5433          AFG      Afghanistan  1960      0.080          -1.123
5434          AFG      Afghanistan  1961      0.083          -1.123
5435          AFG      Afghanistan  1962      0.082          -1.123
5436          AFG      Afghanistan  1963      0.085          -1.123
5437          AFG      Afghanistan  1964      0.137          -0.951
```

1.4 Problem 2

Next focus on the World Bank `wb` dataset 'ESGData.csv'. Use `pandas` methods to perform the following tasks:

1.4.1 Part a

Keep only the columns named 'Country Code', 'Country Name', and 'Indicator Code', or begin with '19' or '20'. (Don't type in all the years individually. Instead, use code that finds all columns that begin '19' or '20'.) [1 point]

```
[9]: cols_to_keep = ['Country Code', 'Country Name', 'Indicator Code'] + \
                    [col for col in wb.columns if col.startswith('19') or col.
                     ↪startswith('20')]
wb_subset = wb[cols_to_keep]

wb_subset.head().T
```

```
[9]:      0      1      2 \
Country Code      ARB      ARB      ARB
Country Name      Arab World      Arab World      Arab World
Indicator Code  EG.CFT.ACCS.ZS  EG.ELC.ACCS.ZS  NY.ADJ.DRES.GN.ZS
1960      NaN      NaN      NaN
1961      NaN      NaN      NaN
...      ...      ...      ...
2019      87.79874      90.352802      7.234436
2020      87.948264      90.63505      4.598506
2021      88.092536      90.845661      NaN
```

2022	NaN	NaN	NaN
2023	NaN	NaN	NaN
	3	4	
Country Code	ARB	ARB	
Country Name	Arab World	Arab World	
Indicator Code	NY.ADJ.DFOR.GN.ZS	AG.LND.AGRI.ZS	
1960	NaN	NaN	
1961	NaN	30.981414	
...	...	...	
2019	0.064516	39.907031	
2020	0.075686	39.97329	
2021	NaN	39.970742	
2022	NaN	NaN	
2023	NaN	NaN	

[67 rows x 5 columns]

1.4.2 Part b

Rename 'Country Code' to 'country_code', 'Country Name' to 'country_name_wb', and 'Indicator Code' to 'feature'. [1 point]

```
[10]: wb_subset = wb_subset.copy()

wb_subset.rename(columns={
    'Country Code': 'country_code',
    'Country Name': 'country_name_wb',
    'Indicator Code': 'feature'
}, inplace=True)

wb_subset.head().T
```

```
[10]:
```

	0	1	2 \
country_code	ARB	ARB	ARB
country_name_wb	Arab World	Arab World	Arab World
feature	EG.CFT.ACCS.ZS	EG.ELC.ACCS.ZS	NY.ADJ.DRES.GN.ZS
1960	NaN	NaN	NaN
1961	NaN	NaN	NaN
...	...	...	...
2019	87.79874	90.352802	7.234436
2020	87.948264	90.63505	4.598506
2021	88.092536	90.845661	NaN
2022	NaN	NaN	NaN
2023	NaN	NaN	NaN
	3	4	
country_code	ARB	ARB	

country_name_wb	Arab World	Arab World
feature	NY.ADJ.DFOR.GN.ZS	AG.LND.AGRI.ZS
1960	NaN	NaN
1961	NaN	30.981414
...	...	...
2019	0.064516	39.907031
2020	0.075686	39.97329
2021	NaN	39.970742
2022	NaN	NaN
2023	NaN	NaN

[67 rows x 5 columns]

1.4.3 Part c

Use the `.query()` method to remove the rows in which 'country_name_wb' is equal to one of the entries in the following `noncountries` list: [1 point]

```
[11]: noncountries = ["Arab World", "Central Europe and the Baltics",
    "Caribbean small states",
    "East Asia & Pacific (excluding high income)",
    "Early-demographic dividend", "East Asia & Pacific",
    "Europe & Central Asia (excluding high income)",
    "Europe & Central Asia", "Euro area",
    "European Union", "Fragile and conflict affected situations",
    "High income",
    "Heavily indebted poor countries (HIPC)", "IBRD only",
    "IDA & IBRD total",
    "IDA total", "IDA blend", "IDA only",
    "Latin America & Caribbean (excluding high income)",
    "Latin America & Caribbean",
    "Least developed countries: UN classification",
    "Low income", "Lower middle income", "Low & middle income",
    "Late-demographic dividend", "Middle East & North Africa",
    "Middle income",
    "Middle East & North Africa (excluding high income)",
    "North America", "OECD members",
    "Other small states", "Pre-demographic dividend",
    "Pacific island small states",
    "Post-demographic dividend",
    "Sub-Saharan Africa (excluding high income)",
    "Sub-Saharan Africa",
    "Small states", "East Asia & Pacific (IDA & IBRD)",
    "Europe & Central Asia (IDA & IBRD)",
    "Latin America & Caribbean (IDA & IBRD)",
    "Middle East & North Africa (IDA & IBRD)", "South Asia",
    "South Asia (IDA & IBRD)",
```

```
"Sub-Saharan Africa (IDA & IBRD)",
"Upper middle income", "World"]
```

```
[12]: wb_subset = wb_subset.query("country_name_wb not in @noncountries")

wb_subset.head().T
```

```
[12]:
```

	3266	3267	3268 \
country_code	AFG	AFG	AFG
country_name_wb	Afghanistan	Afghanistan	Afghanistan
feature	EG.CFT.ACCS.ZS	EG.ELC.ACCS.ZS	NY.ADJ.DRES.GN.ZS
1960	NaN	NaN	NaN
1961	NaN	NaN	NaN
...	...	...	...
2019	32.45	97.7	0.370131
2020	33.8	97.7	0.243668
2021	35.4	97.7	0.335935
2022	NaN	NaN	NaN
2023	NaN	NaN	NaN

	3269	3270
country_code	AFG	AFG
country_name_wb	Afghanistan	Afghanistan
feature	NY.ADJ.DFOR.GN.ZS	AG.LND.AGRI.ZS
1960	NaN	NaN
1961	NaN	57.878356
...	...	...
2019	0.269353	58.276988
2020	0.237958	58.741548
2021	0.317732	58.741548
2022	NaN	NaN
2023	NaN	NaN

```
[67 rows x 5 columns]
```

1.4.4 Part d

The features in this dataset are given strange and incomprehensible codes such as 'EG.CFT.ACCS.ZS'. Use the `replace_map` dictionary, defined below, to recode all of these values with more descriptive names for each feature. [1 point]

```
[13]: replace_map = {
    "AG.LND.AGRI.ZS": "agricultural_land",
    "AG.LND.FRST.ZS": "forest_area",
    "AG.PRD.FOOD.XD": "food_production_index",
    "CC.EST": "control_of_corruption",
    "EG.CFT.ACCS.ZS": "access_to_clean_fuels_and_technologies_for_cooking",
    "EG.EGY.PRIM.PP.KD": "energy_intensity_level_of_primary_energy",
```


"EG.ELC.ACCS.ZS": "access_to_electricity",
 "EG.ELC.COAL.ZS": "electricity_production_from_coal_sources",
 "EG.ELC.RNEW.ZS": "renewable_electricity_output",
 "EG.FEC.RNEW.ZS": "renewable_energy_consumption",
 "EG.IMP.CONNS.ZS": "energy_imports",
 "EG.USE.COMM.FO.ZS": "fossil_fuel_energy_consumption",
 "EG.USE.PCAP.KG.OE": "energy_use",
 "EN.ATM.CO2E.PC": "co2_emissions",
 "EN.ATM.METH.PC": "methane_emissions",
 "EN.ATM.NOXE.PC": "nitrous_oxide_emissions",
 "EN.ATM.PM25.MC.M3": "pm2_5_air_pollution",
 "EN.CLC.CDDY.XD": "cooling_degree_days",
 "EN.CLC.GHGR.MT.CE": "ghg_net_emissions",
 "EN.CLC.HEAT.XD": "heat_index_35",
 "EN.CLC.MDAT.ZS": "droughts",
 "EN.CLC.PRCP.XD": "maximum_5-day_rainfall",
 "EN.CLC.SPEI.XD": "mean_drought_index",
 "EN.MAM.THRD.NO": "mammal_species",
 "EN.POP.DNST": "population_density",
 "ER.H2O.FWTL.ZS": "annual_freshwater_withdrawals",
 "ER.PTD.TOTL.ZS": "terrestrial_and_marine_protected_areas",
 "GB.XPD.RSDV.GD.ZS": "research_and_development_expenditure",
 "GE.EST": "government_effectiveness",
 "IC.BUS.EASE.XQ": "ease_of_doing_business_rank",
 "IC.LGL.CRED.XQ": "strength_of_legal_rights_index",
 "IP.JRN.ARTC.SC": "scientific_and_technical_journal_articles",
 "IP.PAT.RESD": "patent_applications",
 "IT.NET.USER.ZS": "individuals_using_the_internet",
 "NV.AGR.TOTL.ZS": "agriculture",
 "NY.ADJ.DFOR.GN.ZS": "net_forest_depletion",
 "NY.ADJ.DRES.GN.ZS": "natural_resources_depletion",
 "NY.GDP.MKTP.KD.ZG": "gdp_growth",
 "PV.EST": "political_stability_and_absence_of_violence",
 "RL.EST": "rule_of_law",
 "RQ.EST": "regulatory_quality",
 "SE.ADT.LITR.ZS": "literacy_rate",
 "SE.ENR.PRSC.FM.ZS": "gross_school_enrollment",
 "SE.PRM.ENRR": "primary_school_enrollment",
 "SE.XPD.TOTL.GB.ZS": "government_expenditure_on_education",
 "SG.GEN.PARL.ZS": "proportion_of_seats_held_by_women_in_national_parliaments",
 "SH.DTH.COMM.ZS": "cause_of_death",
 "SH.DYN.MORT": "mortality_rate",
 "SH.H2O.SMDW.ZS": "people_using_safely_managed_drinking_water_services",
 "SH.MED.BEDS.ZS": "hospital_beds",
 "SH.STA.OWAD.ZS": "prevalence_of_overweight",
 "SH.STA.SMSS.ZS": "people_using_safely_managed_sanitation_services",
 "SI.DST.FRST.20": "income_share_held_by_lowest_20pct",

```

"SI.POV.GINI": "gini_index",
"SI.POV.NAHC": "poverty_headcount_ratio_at_national_poverty_lines",
"SI.SPR.PCAP.ZG": "\u2191",
"\u2192annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_or_income",
"SL.TLF.0714.ZS": "children_in_employment",
"SL.TLF.ACTI.ZS": "labor_force_participation_rate",
"SL.TLF.CACT.FM.ZS": "ratio_of_female_to_male_labor_force_participation_rate",
"SL.UEM.TOTL.ZS": "unemployment",
"SM.POP.NETM": "net_migration",
"SN.ITK.DEFC.ZS": "prevalence_of_overnourishment",
"SP.DYN.LE00.IN": "life_expectancy_at_birth",
"SP.DYN.TFRT.IN": "fertility_rate",
"SP.POP.65UP.TO.ZS": "population_ages_65_and_above",
"SP.UWT.TFRT": "unmet_need_for_contraception",
"VA.EST": "voice_and_accountability",
"EN.CLC.CSTP.ZS": "coastal_protection",
"SD.ESR.PERF.XQ": "economic_and_social_rights_performance_score",
"EN.CLC.HDDY.XD": "heating_degree_days",
"EN.LND.LTMP.DC": "land_surface_temperature",
"ER.H2O.FWST.ZS": "freshwater_withdrawal",
"EN.H2O.BDYS.ZS": "water_quality",
"AG.LND.FRLS.HA": "tree_cover_loss",
}

```

```

[14]: wb_subset['feature'] = wb_subset['feature'].replace(replace_map)

wb_subset.head().T

```

```

[14]:
country_code      3266 \
country_name_wb    AFG
feature           access_to_clean_fuels_and_technologies_for_coo...
1960              NaN
1961              NaN
...              ...
2019             32.45
2020             33.8
2021             35.4
2022             NaN
2023             NaN

country_code      3267      3268 \
country_name_wb    AFG      AFG
country_name_wb    Afghanistan  Afghanistan
feature           access_to_electricity  natural_resources_depletion
1960              NaN              NaN
1961              NaN              NaN

```

```

...
2019          97.7          0.370131
2020          97.7          0.243668
2021          97.7          0.335935
2022          NaN          NaN
2023          NaN          NaN

          3269          3270
country_code          AFG          AFG
country_name_wb      Afghanistan      Afghanistan
feature      net_forest_depletion      agricultural_land
1960          NaN          NaN
1961          NaN          57.878356
...
2019          0.269353          58.276988
2020          0.237958          58.741548
2021          0.317732          58.741548
2022          NaN          NaN
2023          NaN          NaN

```

[67 rows x 5 columns]

1.5 Problem 3

The `wb` dataset is strangely organized. The features are stored in the rows, when typically we would want these features to be columns. Also, years are stored in columns, when typically we would want years to be represented by different rows. We can repair this structure by reshaping the data.

1.5.1 Part a

First, reshape the data to turn the columns that refer to years into rows. [1 point]

```

[15]: wb_melted = pd.melt(
        wb_subset,
        id_vars=['country_code', 'country_name_wb', 'feature'],
        value_name='value'
    )

wb_melted.head()

```

```

[15]: country_code country_name_wb \
0      AFG      Afghanistan
1      AFG      Afghanistan
2      AFG      Afghanistan
3      AFG      Afghanistan
4      AFG      Afghanistan

```

```

feature variable  value

```

0	access_to_clean_fuels_and_technologies_for_coo...	1960	NaN
1	access_to_electricity	1960	NaN
2	natural_resources_depletion	1960	NaN
3	net_forest_depletion	1960	NaN
4	agricultural_land	1960	NaN

1.5.2 Part b

Then rename `variable` to `year`, and reshape the data again by turning the rows that refer to features into columns. [1 point]

```
[16]: wb_melted.rename(columns={'variable': 'year'}, inplace=True)

wb_resaped = wb_melted.pivot_table(
    index=['country_code', 'country_name_wb', 'year'],
    columns='feature',
    values='value'
).reset_index()

wb_resaped.head().T
```

```
[16]:
```

	0	1 \
feature		
country_code	AFG	AFG
country_name_wb	Afghanistan	Afghanistan
year	1960	1961
access_to_clean_fuels_and_technologies_for_cooking	NaN	NaN
access_to_electricity	NaN	NaN
...	...	...
tree_cover_loss	NaN	NaN
unemployment	NaN	NaN
unmet_need_for_contraception	NaN	NaN
voice_and_accountability	NaN	NaN
water_quality	NaN	NaN

	2	3 \
feature		
country_code	AFG	AFG
country_name_wb	Afghanistan	Afghanistan
year	1962	1963
access_to_clean_fuels_and_technologies_for_cooking	NaN	NaN
access_to_electricity	NaN	NaN
...	...	...
tree_cover_loss	NaN	NaN
unemployment	NaN	NaN
unmet_need_for_contraception	NaN	NaN
voice_and_accountability	NaN	NaN
water_quality	NaN	NaN

```

feature
country_code                AFG
country_name_wb             Afghanistan
year                        1964
access_to_clean_fuels_and_technologies_for_cooking    NaN
access_to_electricity       NaN
...                          ...
tree_cover_loss             NaN
unemployment                NaN
unmet_need_for_contraception    NaN
voice_and_accountability     NaN
water_quality               NaN

[74 rows x 5 columns]

```

1.5.3 Part c

After these reshapes, the year column in the `wb` data frame is stored as a string. Convert this column to an integer data type. [1 point]

```
[17]: wb_resaped['year'] = wb_resaped['year'].astype(int)

print(wb_resaped['year'].dtype)
```

```
int64
```

1.6 Problem 4

Next we will merge the `wb` data frame with the `vdem` data frame, matching on the ‘country_code’ and ‘year’ columns.

1.6.1 Part a

First, write a sentence stating whether you expect this merge to be one-to-one, many-to-one, one-to-many, or many-to-many, and describe your rationale. [1 point]

I expect this merge to be one-to-one because both datasets have been structured so that each row represents a unique country-year observation. In the V-Dem dataset, every row corresponds to one country in a given year, and after reshaping, the World Bank data also has one row per country-year combination.

1.6.2 Part b

Next, merge the two datasets together in a way that checks whether your expectation is met, and also allows you to see the rows that failed to match. [2 points]

```
[18]: merged = pd.merge(
        wb_resshaped,
        vdem_subset,
        on=['country_code', 'year'],
        how='outer',
        validate= 'one_to_one',
        indicator=True
    )

merged.head()
```

```
[18]:   country_code country_name_wb  year \
0          AFG    Afghanistan  1960
1          AFG    Afghanistan  1961
2          AFG    Afghanistan  1962
3          AFG    Afghanistan  1963
4          AFG    Afghanistan  1964

      access_to_clean_fuels_and_technologies_for_cooking  access_to_electricity \
0                                     NaN                                     NaN
1                                     NaN                                     NaN
2                                     NaN                                     NaN
3                                     NaN                                     NaN
4                                     NaN                                     NaN

      agricultural_land  agriculture  annual_freshwater_withdrawals \
0                NaN          NaN          NaN
1        57.878356          NaN          NaN
2        57.955016          NaN          NaN
3        58.031676          NaN          NaN
4        58.116002          NaN          NaN

      annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_or_
income \
0                NaN
1                NaN
2                NaN
3                NaN
4                NaN

      cause_of_death  ...  terrestrial_and_marine_protected_areas \
0                NaN  ...                NaN
1                NaN  ...                NaN
2                NaN  ...                NaN
3                NaN  ...                NaN
4                NaN  ...                NaN
```

	tree_cover_loss	unemployment	unmet_need_for_contraception	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	

	voice_and_accountability	water_quality	country_name_vdem	democracy	\
0	NaN	NaN	Afghanistan	0.080	
1	NaN	NaN	Afghanistan	0.083	
2	NaN	NaN	Afghanistan	0.082	
3	NaN	NaN	Afghanistan	0.085	
4	NaN	NaN	Afghanistan	0.137	

	educational_equality	_merge
0	-1.123	both
1	-1.123	both
2	-1.123	both
3	-1.123	both
4	-0.951	both

[5 rows x 78 columns]

```
[19]: unmatched_rows = merged[merged['_merge'] != 'both']
print(unmatched_rows.head())
```

	country_code	country_name_wb	year	\
62	AFG	Afghanistan	2022	
63	AFG	Afghanistan	2023	
126	AGO	Angola	2022	
127	AGO	Angola	2023	
190	ALB	Albania	2022	

	access_to_clean_fuels_and_technologies_for_cooking	\
62	NaN	
63	NaN	
126	NaN	
127	NaN	
190	NaN	

	access_to_electricity	agricultural_land	agriculture	\
62	NaN	NaN	NaN	
63	NaN	NaN	NaN	
126	NaN	NaN	13.649109	
127	NaN	NaN	NaN	
190	NaN	NaN	18.622821	

	annual_freshwater_withdrawals	\
--	-------------------------------	---

62	NaN
63	NaN
126	NaN
127	NaN
190	NaN

annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_o	
r_income \	
62	NaN
63	NaN
126	NaN
127	NaN
190	NaN

cause_of_death ... terrestrial_and_marine_protected_areas \		
62	NaN ...	3.637257
63	NaN ...	NaN
126	NaN ...	5.004710
127	NaN ...	NaN
190	NaN ...	14.175197

tree_cover_loss unemployment unmet_need_for_contraception \			
62	NaN	14.100	NaN
63	NaN	15.378	NaN
126	NaN	14.478	NaN
127	NaN	14.223	NaN
190	NaN	11.629	NaN

voice_and_accountability water_quality country_name_vdem democracy \				
62	-1.751587	NaN	NaN	NaN
63	NaN	NaN	NaN	NaN
126	-0.797131	NaN	NaN	NaN
127	NaN	NaN	NaN	NaN
190	0.139466	NaN	NaN	NaN

educational_equality _merge	
62	NaN left_only
63	NaN left_only
126	NaN left_only
127	NaN left_only
190	NaN left_only

[5 rows x 78 columns]

1.6.3 Part c

After this merge, use the `.value_counts()` method to see the total number of observations that were found in both datasets, the number found only in the left dataset, and the number found

only in the right dataset. (If you entered the `wb` data frame into the merge function first, then “left_only” refers to the rows found in the World Bank but not V-Dem, and “right_only” refers to the rows found in V-Dem but not the World Bank.) There should be more than 9000 rows that matched, but more than 2000 that failed to match.

Then conduct two data aggregations to help us investigate why these observations did not match:

- First use `.query()` to keep only the observations that were present in `wb` but not `vdem`. (These are the ‘left_only’ observations if you typed the World Bank data into the merge function first.) Use `.groupby()` to aggregate the data by both ‘country_code’ and ‘country_name_wb’. Then save the minimum and maximum values of ‘year’ for each country.
- Then use `.query()` to keep only the observations that were present in `vdem` data but not `wb`. Use `.groupby()` to aggregate the data by both ‘country_code’ and ‘country_name_vdem’. Then save the minimum and maximum values of ‘year’ for each country. [2 points]

```
[20]: # left_only is wb and right_only is vdem
merge_counts = merged['_merge'].value_counts()
print("Merge counts:")
print(merge_counts)
```

Merge counts:

```
_merge
both          9976
left_only     2360
right_only     395
Name: count, dtype: int64
```

```
[21]: pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

left_only = merged.query("_merge == 'left_only'")
left_grouped = left_only.groupby(['country_code', 'country_name_wb'])['year'].
    .agg(['min', 'max']).reset_index()

print(left_grouped.to_string())

pd.reset_option('display.max_rows')
pd.reset_option('display.max_columns')
```

	country_code	country_name_wb	min	max
0	AFG	Afghanistan	2022	2023
1	AGO	Angola	2022	2023
2	ALB	Albania	2022	2023
3	AND	Andorra	1960	2022
4	ARE	United Arab Emirates	1960	2023
5	ARG	Argentina	2022	2023
6	ARM	Armenia	1960	2023
7	ATG	Antigua and Barbuda	1960	2022
8	AUS	Australia	2022	2023

9	AUT	Austria	2022	2023
10	AZE	Azerbaijan	1960	2023
11	BDI	Burundi	2022	2023
12	BEL	Belgium	2022	2023
13	BEN	Benin	2022	2023
14	BFA	Burkina Faso	2022	2023
15	BGD	Bangladesh	1960	2023
16	BGR	Bulgaria	2022	2023
17	BHR	Bahrain	2022	2023
18	BHS	Bahamas, The	1960	2023
19	BIH	Bosnia and Herzegovina	1960	2023
20	BLR	Belarus	1960	2023
21	BLZ	Belize	1960	2023
22	BOL	Bolivia	2022	2023
23	BRA	Brazil	2022	2023
24	BRB	Barbados	2022	2023
25	BRN	Brunei Darussalam	1960	2023
26	BTN	Bhutan	2022	2023
27	BWA	Botswana	2022	2023
28	CAF	Central African Republic	2022	2023
29	CAN	Canada	2022	2023
30	CHE	Switzerland	2022	2023
31	CHL	Chile	2022	2023
32	CHN	China	2022	2023
33	CIV	Cote d'Ivoire	2022	2023
34	CMR	Cameroon	1960	2023
35	COD	Congo, Dem. Rep.	2022	2023
36	COG	Congo, Rep.	2022	2023
37	COL	Colombia	2022	2023
38	COM	Comoros	2022	2023
39	CPV	Cabo Verde	2022	2023
40	CRI	Costa Rica	2022	2023
41	CUB	Cuba	2022	2023
42	CYP	Cyprus	2022	2023
43	CZE	Czechia	2022	2023
44	DEU	Germany	2022	2023
45	DJI	Djibouti	2022	2023
46	DMA	Dominica	1960	2022
47	DNK	Denmark	2022	2023
48	DOM	Dominican Republic	2022	2023
49	DZA	Algeria	2022	2023
50	ECU	Ecuador	2022	2023
51	EGY	Egypt, Arab Rep.	2022	2023
52	ERI	Eritrea	2022	2023
53	ESP	Spain	2022	2023
54	EST	Estonia	1960	2023
55	ETH	Ethiopia	2022	2023
56	FIN	Finland	2022	2023

57	FJI	Fiji	2022	2023
58	FRA	France	2022	2023
59	FSM	Micronesia, Fed. Sts.	1960	2022
60	GAB	Gabon	2022	2023
61	GBR	United Kingdom	2022	2023
62	GEO	Georgia	1960	2023
63	GHA	Ghana	2022	2023
64	GIN	Guinea	2022	2023
65	GMB	Gambia, The	2022	2023
66	GNB	Guinea-Bissau	2022	2023
67	GNQ	Equatorial Guinea	2022	2023
68	GRC	Greece	2022	2023
69	GRD	Grenada	1960	2022
70	GTM	Guatemala	2022	2023
71	GUY	Guyana	2022	2023
72	HND	Honduras	2022	2023
73	HRV	Croatia	1960	2023
74	HTI	Haiti	2022	2023
75	HUN	Hungary	2022	2023
76	IDN	Indonesia	2022	2023
77	IND	India	2022	2023
78	IRL	Ireland	2022	2023
79	IRN	Iran, Islamic Rep.	2022	2023
80	IRQ	Iraq	2022	2023
81	ISL	Iceland	2022	2023
82	ISR	Israel	2022	2023
83	ITA	Italy	2022	2023
84	JAM	Jamaica	2022	2023
85	JOR	Jordan	2022	2023
86	JPN	Japan	2022	2023
87	KAZ	Kazakhstan	1960	2023
88	KEN	Kenya	2022	2023
89	KGZ	Kyrgyz Republic	1960	2023
90	KHM	Cambodia	2022	2023
91	KIR	Kiribati	1960	2022
92	KNA	St. Kitts and Nevis	1960	2022
93	KOR	Korea, Rep.	2022	2023
94	KWT	Kuwait	2022	2023
95	LAO	Lao PDR	2022	2023
96	LBN	Lebanon	2022	2023
97	LBR	Liberia	2022	2023
98	LBY	Libya	2022	2023
99	LCA	St. Lucia	1960	2023
100	LIE	Liechtenstein	1960	2022
101	LKA	Sri Lanka	2022	2023
102	LSO	Lesotho	2022	2023
103	LTU	Lithuania	1960	2023
104	LUX	Luxembourg	2022	2023

105	LVA	Latvia	1960	2023
106	MAR	Morocco	2022	2023
107	MCO	Monaco	1960	2022
108	MDA	Moldova	1960	2023
109	MDG	Madagascar	2022	2023
110	MDV	Maldives	2022	2023
111	MEX	Mexico	2022	2023
112	MHL	Marshall Islands	1960	2022
113	MKD	North Macedonia	1960	2023
114	MLI	Mali	2022	2023
115	MLT	Malta	2022	2023
116	MMR	Myanmar	2022	2023
117	MNE	Montenegro	1960	2023
118	MNG	Mongolia	2022	2023
119	MOZ	Mozambique	2022	2023
120	MRT	Mauritania	2022	2023
121	MUS	Mauritius	2022	2023
122	MWI	Malawi	2022	2023
123	MYS	Malaysia	2022	2023
124	NAM	Namibia	2022	2023
125	NER	Niger	2022	2023
126	NGA	Nigeria	2022	2023
127	NIC	Nicaragua	2022	2023
128	NLD	Netherlands	2022	2023
129	NOR	Norway	2022	2023
130	NPL	Nepal	2022	2023
131	NRU	Nauru	1960	2022
132	NZL	New Zealand	2022	2023
133	OMN	Oman	2022	2023
134	PAK	Pakistan	2022	2023
135	PAN	Panama	2022	2023
136	PER	Peru	2022	2023
137	PHL	Philippines	2022	2023
138	PLW	Palau	1960	2022
139	PNG	Papua New Guinea	2022	2023
140	POL	Poland	2022	2023
141	PRK	Korea, Dem. People's Rep.	2022	2023
142	PRT	Portugal	2022	2023
143	PRY	Paraguay	2022	2023
144	QAT	Qatar	2022	2023
145	ROU	Romania	2022	2023
146	RUS	Russian Federation	2022	2023
147	RWA	Rwanda	2022	2023
148	SAU	Saudi Arabia	2022	2023
149	SDN	Sudan	2022	2023
150	SEN	Senegal	2022	2023
151	SGP	Singapore	2022	2023
152	SLB	Solomon Islands	2022	2023

153	SLE	Sierra Leone	2022	2023
154	SLV	El Salvador	2022	2023
155	SMR	San Marino	1960	2022
156	SOM	Somalia	2022	2023
157	SRB	Serbia	2022	2023
158	SSD	South Sudan	1960	2023
159	STP	Sao Tome and Principe	2022	2023
160	SUR	Suriname	2022	2023
161	SVK	Slovak Republic	1960	2023
162	SVN	Slovenia	1960	2023
163	SWE	Sweden	2022	2023
164	SWZ	Eswatini	2022	2023
165	SYC	Seychelles	2022	2022
166	SYR	Syrian Arab Republic	2022	2023
167	TCD	Chad	2022	2023
168	TGO	Togo	2022	2023
169	THA	Thailand	2022	2023
170	TJK	Tajikistan	1960	2023
171	TKM	Turkmenistan	1960	2023
172	TLS	Timor-Leste	2022	2023
173	TON	Tonga	1960	2023
174	TTO	Trinidad and Tobago	2022	2023
175	TUN	Tunisia	2022	2023
176	TUR	Turkiye	2022	2023
177	TUV	Tuvalu	1960	2022
178	TZA	Tanzania	2022	2023
179	UGA	Uganda	2022	2023
180	UKR	Ukraine	1960	2022
181	URY	Uruguay	2022	2023
182	USA	United States	2022	2023
183	UZB	Uzbekistan	1960	2023
184	VCT	St. Vincent and the Grenadines	1960	2023
185	VEN	Venezuela, RB	2022	2023
186	VNM	Viet Nam	2022	2023
187	VUT	Vanuatu	2022	2023
188	WSM	Samoa	1960	2023
189	YEM	Yemen, Rep.	2022	2023
190	ZAF	South Africa	2022	2023
191	ZMB	Zambia	2022	2023
192	ZWE	Zimbabwe	2022	2023

```
[22]: right_only = merged.query("_merge == 'right_only'")
right_grouped = right_only.groupby(['country_code', ↵
↳ 'country_name_vdem'])['year'].agg(['min', 'max']).reset_index()
right_grouped
```

```
[22]:
```

	country_code		country_name_vdem	min	max
0	DDR	German	Democratic Republic	1960	1990
1	HKG		Hong Kong	1960	2021
2	PSE		Palestine/West Bank	1967	2021
3	PSG		Palestine/Gaza	1960	2021
4	SML		Somaliland	1991	2021
5	TWN		Taiwan	1960	2021
6	VDR		Republic of Vietnam	1960	1975
7	KKX		Kosovo	1999	2021
8	YMD		South Yemen	1960	1990
9	ZZB		Zanzibar	1960	2021

1.6.4 Part d

Here's where a deep understanding of the data becomes very important. There are two reasons why an observation may fail to match in a merge. One reason is a difference in spelling. Suppose that South Korea (which is also known as the Republic of Korea) is coded as SKO in the World Bank data and ROK in V-Dem. In this case, we should recode one or the other of SKO and ROK so that they match, otherwise we will lose the data on South Korea. But the second reason why observations might fail to match is due to differences in coverage in the data collection strategy: it is possible that a country wasn't included in one data's coverage, or that certain years for that country were not included. For differences in coverage, there's no way to manipulate the data to match, so we are out of luck and we have to either delete these observations or proceed with missing data from one of the data sources.

Take a close look at the two data aggregation tables you generated in part (j), and answer the following questions:

- Do you see any countries that are present in both the unmatched World Bank rows and the unmatched V-Dem rows, but with different spellings?
- Do some digging on Wikipedia and other sources on the Internet. What do you think is the primary reason why some countries are present in the V-Dem data but not the World Bank? (You don't need to describe the reasoning for every country. Just dig until you see a general pattern and describe it here.)
- Do some more digging on Wikipedia and other sources on the Internet. What do you think is the primary reason why some countries are present in the World Bank data but not V-Dem? (You don't need to describe the reasoning for every country. Just dig until you see a general pattern and describe it here.) [1 point]

Different Spellings - I did not find any obvious cases where the same country appears in both the unmatched World Bank and V-Dem groups with just a difference in spelling. The unmatched observations in each set seem to be driven more by differences in the data's scope rather than simple naming inconsistencies.

V-Dem but Not World Bank: - Many of the unmatched observations in the V-Dem data (right_only) are historical or disputed entities—for example, the German Democratic Republic (DDR), South Yemen, Somaliland, Taiwan, and Kosovo. These entities either no longer exist as separate states or have contested international status. The World Bank generally includes only

those countries that are recognized as sovereign states by most international organizations, so entities with ambiguous or historical status tend not to appear in its data.

World Bank but Not V-Dem: - The unmatched left_only observations in the World Bank data tend to show a minimum and maximum year of 2022–2023, which indicates that for those years, the data is available only in the World Bank dataset and not in V-Dem. This suggests that the World Bank dataset is more current—extending to 2023—while the V-Dem dataset stops at 2021, resulting in a mismatch for these recent years.

1.6.5 Part e

Once you are convinced that all of the unmatched observations are due to differences in the coverage of the data collection strategies of the World Bank and V-Dem, repeat the merge, dropping all unmatched observations. This time there is no need to validate the type of merge, and no need to define a variable to indicate matching. [1 point]

```
[23]: merged_inner = pd.merge(wb_resshaped, vdem_subset, on=['country_code', 'year'],
    ↪how='inner')

merged_inner.head()
```

```
[23]:   country_code country_name_wb  year \
0          AFG    Afghanistan  1960
1          AFG    Afghanistan  1961
2          AFG    Afghanistan  1962
3          AFG    Afghanistan  1963
4          AFG    Afghanistan  1964

      access_to_clean_fuels_and_technologies_for_cooking  access_to_electricity \
0                                     NaN                                     NaN
1                                     NaN                                     NaN
2                                     NaN                                     NaN
3                                     NaN                                     NaN
4                                     NaN                                     NaN

      agricultural_land  agriculture  annual_freshwater_withdrawals \
0                NaN          NaN          NaN
1        57.878356          NaN          NaN
2        57.955016          NaN          NaN
3        58.031676          NaN          NaN
4        58.116002          NaN          NaN

      annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_or_
income \
0                NaN
1                NaN
2                NaN
3                NaN
```

```

4                                     NaN

   cause_of_death ... strength_of_legal_rights_index \
0               NaN ...                               NaN
1               NaN ...                               NaN
2               NaN ...                               NaN
3               NaN ...                               NaN
4               NaN ...                               NaN

   terrestrial_and_marine_protected_areas  tree_cover_loss  unemployment \
0                                     NaN                NaN                NaN
1                                     NaN                NaN                NaN
2                                     NaN                NaN                NaN
3                                     NaN                NaN                NaN
4                                     NaN                NaN                NaN

   unmet_need_for_contraception  voice_and_accountability  water_quality \
0                               NaN                      NaN                NaN
1                               NaN                      NaN                NaN
2                               NaN                      NaN                NaN
3                               NaN                      NaN                NaN
4                               NaN                      NaN                NaN

   country_name_vdem  democracy  educational_equality
0      Afghanistan    0.080        -1.123
1      Afghanistan    0.083        -1.123
2      Afghanistan    0.082        -1.123
3      Afghanistan    0.085        -1.123
4      Afghanistan    0.137        -0.951

```

[5 rows x 77 columns]

1.7 Problem 5

Write code using **pandas** that answers the next two questions:

1.7.1 Part a

Of all countries in the data, which countries have the highest and lowest average levels of democratic quality across the 1960-2022 timespan? [1 point]

```

[24]: avg_democracy = merged_inner.groupby(['country_code',
      ↪ 'country_name_vdem'])['democracy'].mean().reset_index()

highest_democracy = avg_democracy.loc[avg_democracy['democracy'].idxmax()]

lowest_democracy = avg_democracy.loc[avg_democracy['democracy'].idxmin()]

```



```
print("Country with Highest Average Democracy Level:")
print(highest_democracy)

print("\nCountry with Lowest Average Democracy Level:")
print(lowest_democracy)
```

Country with Highest Average Democracy Level:

```
country_code      DNK
country_name_vdem  Denmark
democracy          0.910452
Name: 41, dtype: object
```

Country with Lowest Average Democracy Level:

```
country_code      SAU
country_name_vdem  Saudi Arabia
democracy          0.014806
Name: 132, dtype: object
```

1.7.2 Part b

The ‘educational_equality’ index compiled by V-Dem measures the extent to which “high quality basic education guaranteed to all, sufficient to enable them to exercise their basic rights as adult citizens.” They use a Bayesian scaling method to create a score for each country in each year that ranges roughly from -4 to 4, where low values of the scale mean that > Provision of high quality basic education is extremely unequal and at least 75 percent (%) of children receive such low-quality education that undermines their ability to exercise their basic rights as adult citizens.

And high values mean that > Basic education is equal in quality and less than five percent (%) of children receive such low-quality education that probably undermines their ability to exercise their basic rights as adult citizens.

Use the `pd.cut()` method to create a categorical version of ‘educational_equality’ with five categories, one from -4 to -2 called “extremely unequal”, one from -2 to -.5 called “very unequal”, one from -.5 to .5 called “somewhat unequal”, one from .5 to 1.5 called “relatively equal”, and one for values from 1.5 to 4 called “equal”. (By default, the `pd.cut()` method sets `right=True`, which means the bins include their rightmost edges, so a value of exactly -2 will fall within the “extremely unequal” bin. Leave this default in place.)

Then aggregate the data to have one row per category of the new categorical version of “educational_equality”. Collapse the following features to the mean with each category of “educational_equality”:

- ‘gini_index’: The GINI index measures the amount of economic inequality in a country. The higher the index, the greater the economic disparity between rich and poor.
- ‘poverty_headcount_ratio_at_national_poverty_lines’: a measure of the proportion of the population living in poverty [1 point]

```
[25]: bins = [-4, -2, -0.5, 0.5, 1.5, 4]
labels = ["extremely unequal", "very unequal", "somewhat unequal", "relatively_
↪equal", "equal"]
```

```
merged_inner['educational_equality_cat'] = pd.
    ↪cut(merged_inner['educational_equality'], bins=bins, labels=labels)

aggregated = merged_inner.groupby('educational_equality_cat', observed=False)[
    ['gini_index', 'poverty_headcount_ratio_at_national_poverty_lines']
].mean().reset_index()

aggregated
```

```
[25]: educational_equality_cat  gini_index  \
0      extremely unequal      38.846154
1      very unequal          45.926484
2      somewhat unequal      43.200442
3      relatively equal       37.148861
4      equal                 32.652901

poverty_headcount_ratio_at_national_poverty_lines
0      58.160000
1      38.636058
2      24.149123
3      22.548536
4      17.207444
```